



AuthN and AuthZ (part 1/2)

September 2026

Recap day 3



Last time: API design principles, naming, nesting, compatibility



We looked at contract quality and integration testing



The project review focused on API quality and testing



Today: the next required layer of a real API → security

Today's agenda



Authentication vs. authorization



401 vs. 403



Basic auth and role checks



Resource ownership and IDOR



Security tests for API behavior



Section 1

Authentication vs. authorization

The API must know who is calling



Authentication answers: who are you?



Authorization answers: what are you allowed to do?



A user can be known and still be forbidden



Security is part of the API contract

Why this matters so much in APIs

- APIs are called by browsers, apps, and other services
- No natural browser session exists in a raw API call
- UI hiding a button is not real security
- The backend must enforce the rule



401 vs. 403



401 Unauthorized: not authenticated



403 Forbidden: authenticated, but not allowed



The distinction is critical for clients



Use the most specific status code

Basic auth in concept

- Basic auth sends username and password in the header
- Not ideal for production without HTTPS
- Good educational model, not the final answer



API keys, tokens and JWTs



API key

Good for services and machine-to-machine access



Bearer token

Proves possession of a token



JWT

Token with claims, often used for user identity

Mini exercise

AuthN and AuthZ debugging

- Look at several scenarios
- Decide whether it is authentication or authorization
- Fix it with the best status code and expected behavior





Section 2

Role checks and resource ownership

Role-based access control



Roles group users by permission level



Example: USER, ADMIN, MANAGER



Endpoints can require certain roles



Good for simple, predictable permission models

Resource-based authorization

- Not only: "what is the role?"
- Also: "What resource is being accessed?"
- Example:
 - A user can view their own profile only
- Ownership and scope must be enforced by the API



IDOR: a common and dangerous bug



Insecure Direct Object Reference



Direct resource ID without authorization check



Example: user 9 requests `/users/7`

Where to enforce authorization

- Endpoint checks
- Service layer checks
- Data layer checks
- UI checks are never enough



Mini exercise

Fix the authorization bug

- Inspect a small app
- Find the broken auth
- Fix it



CORS and browser security



CORS controls browser cross-origin calls



The browser can block a request even when the backend works



CORS is not the same as API authentication



Keep it explicit and minimal

Mini exercise

Fun with CORS

- The API works with curl, postman, etc
- The browser blocks the frontend request
- Diagnose and understand the behavior





Section 3

Security testing

Why security needs separate testing



Happy path tests do not cover unauthorized access



Security bugs hide in edge cases



Real APIs must be tested as real systems



What to test in practice

- No credentials → 401
- Valid login, wrong permissions → 403
- Wrong user ID → forbidden or not found, by contract
- Valid request → 200 or 201

Rest Assured for security testing

- Send requests without auth
- Send requests with wrong or expired tokens
- Validate response status and body
- Confirm the contract remains consistent



Follow-along demo

Security tests in practice

- GET profile without token
- GET admin as normal user
- GET other user's resource
- GET profile with valid token



Mini exercise

Security tests

- Design tests for authentication and authorization boundaries.





Summary



Authentication vs. authorization



401 vs. 403



Basic auth and roles



Resource ownership and IDOR



Security tests are part of the API contract

Project focus for the afternoon

- Add auth to one protected route
- Add role or ownership checks
- Write at least one unauthorized test
- Review API access boundaries